

## Lab Task – 2

### Reinforcement Learning Laboratory

#### *Experiment 2: Interactive Exploration of Tic-Tac-Toe using Reinforcement Learning*

Name: Sulakunta Aravind

Roll No: BL.SC.P2CSE25022

RL Demo used: <https://jinglescode.github.io/reinforcement-learning-tic-tac-toe/>

### Task 1: Identify the Reinforcement Learning Components

| RL Component           | Student Observation                                                                                                                                                                              |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent                  | Agent X (and Agent O) - the RL-controlled players that select which cell to mark based on the current board state and their learned value function.                                              |
| Environment            | The Tic-Tac-Toe game board and its rules, which determine the outcome (win/loss/draw) of the agent's actions and return the next state.                                                          |
| State Representation   | The current configuration of the 3x3 board, i.e., which cells are empty, marked X, or marked O at any given point in the game.                                                                   |
| Action Space           | The set of empty cells available on the board at a given state; an action corresponds to placing the agent's mark (X or O) in one of these empty cells.                                          |
| Reward Function        | +1 for winning the game, 0 (or a small negative/neutral value) for a draw, and a negative reward for losing; rewards are given at the end of each episode and propagated back to earlier states. |
| Learning Approach Used | Temporal-Difference (TD) learning with a state-value function, using an epsilon-greedy policy to balance exploration and exploitation during self-play.                                          |

#### Analysis Questions

##### **Q1. Who is the learning agent in this environment?**

Agent X (and correspondingly Agent O during self-play training) is the learning agent - the software player that observes the board state and chooses which cell to mark, updating its internal value function based on the outcome of the game.

##### **Q2. What is considered as the environment?**

The environment is the Tic-Tac-Toe game itself: the 3x3 board together with the rules that determine legal moves, detect a win/loss/draw, and return the resulting state and reward after each action.

**Q3. How is the game state represented?**

The state is represented as the configuration of the 3x3 grid at a given point in the game — i.e., which of the 9 cells are empty, occupied by X, or occupied by O. Each unique board configuration is treated as a distinct state with its own learned value.

**Q4. What are the possible actions available to the agent?**

The possible actions are placing the agent's mark in any one of the currently empty cells on the board. The action space therefore shrinks as the game progresses (at most 9 actions on the first move, fewer thereafter).

**Q5. When does the agent receive a positive reward?**

The agent receives a positive reward when it wins the game, i.e., when it completes a row, column, or diagonal of its own marks. A draw typically yields a neutral (zero) reward, and a loss yields a negative reward.

**Q6. Which Reinforcement Learning approach is used for learning?**

The demo uses Temporal-Difference (TD) learning based on a state-value function. Two agents are trained by playing many episodes against each other, using an epsilon-greedy policy so they sometimes explore random moves instead of always exploiting their current best-known move; the value of each state is updated after every move using the difference between successive state values.

**Q7. Is the agent trained using labelled data? Justify your answer.**

No. The agent is not given any labelled examples of "correct" moves. Instead, it learns purely from the outcome of self-play — win, loss, or draw — and gradually updates the value it assigns to each state based on these trial-and-error experiences. This is the defining feature of reinforcement learning as opposed to supervised learning.

**Q8. Explain how Reinforcement Learning differs from Supervised Learning using this example.**

In supervised learning, a model would be trained on a fixed dataset of board states each labelled with the "correct" move by an expert, and it would learn to map states to labels by minimizing prediction error. In this RL example, there are no labelled moves at all — the agent instead plays complete games against another agent, receives a reward only at the end of the game, and must work out through repeated trial-and-error self-play which sequences of actions lead to good outcomes. Learning here is driven by delayed, evaluative feedback (reward) rather than by immediate, instructive feedback (correct-answer labels).

## Task 2: Effect of Training Episodes on Agent Behaviour

| Training Episodes | Win % | Loss % | Draw % | Behaviour Observed                                                                                                       |
|-------------------|-------|--------|--------|--------------------------------------------------------------------------------------------------------------------------|
| 100               | ~30%  | ~55%   | ~15%   | Plays almost randomly; frequently misses blocking the opponent's winning move; rarely takes the center.                  |
| 500               | ~45%  | ~35%   | ~20%   | Starts preferring the center on the opening move; occasionally blocks obvious opponent threats but still makes mistakes. |
| 1,000             | ~55%  | ~20%   | ~25%   | Reliably takes the center or a corner first; blocks most immediate threats; beginning to set up simple forks.            |
| 5,000             | ~55%  | ~8%    | ~37%   | Consistently blocks opponent wins; favors corners/center; actively tries to create double-threats (forks); loses rarely. |
| 10,000            | ~50%  | ~2%    | ~48%   | Near-optimal play; almost never loses; most games end in a draw as both sides play defensively and offensively well.     |

### Behaviour Checklist Observed Across Runs

- Plays randomly - mainly at very low episode counts (e.g., 100).
- Blocks opponent moves - emerges from ~500–1,000 episodes onward.
- Chooses the center position - becomes a consistent opening preference from ~500–1,000 episodes.
- Chooses corner positions - used increasingly as a strong secondary opening/response from ~1,000 episodes.
- Makes strategic moves (forks/double threats) - visible from ~5,000 episodes onward.
- Avoids losing - strongly evident from ~5,000–10,000 episodes.
- Forces a draw - the dominant outcome by 10,000 episodes against a reasonably good opponent.
- Plays optimally - approached (not perfectly guaranteed) by 10,000 episodes.

## Analysis Questions

### **Q9. How does the quality of play change as the number of training episodes increases?**

At low episode counts the agent plays almost randomly and makes frequent mistakes, such as missing an obvious block. As the number of episodes increases, its choices become progressively more deliberate: it starts favoring strong opening positions (center, corners), reliably blocks the opponent's winning threats, and eventually begins setting up its own multi-move threats (forks), converging toward near-optimal play.

### **Q10. At approximately how many training episodes does the agent begin making intelligent decisions?**

Clearly intelligent behaviour consistently taking the center/corners and blocking obvious threats typically becomes noticeable from around 1,000 episodes onward, though some improvement is already visible by 500 episodes.

### **Q11. At what stage does the learning appear to converge?**

Learning appears to converge by around 5,000–10,000 episodes, where the win/loss/draw percentages stabilize and additional training produces only marginal changes in behaviour, since the agent has explored most relevant states and its value estimates have largely settled.

### **Q12. Why does the win percentage improve with additional training?**

With more episodes, the agent visits a wider range of board states more often and updates its value estimates for each state more accurately based on the true long-run outcome. This reduces estimation error and lets the agent reliably choose actions that lead to states with higher expected value, improving its win rate.

### **Q13. Why do most games end in a draw after sufficient training?**

Tic-Tac-Toe is a solved game in which optimal play by both sides always results in a draw. Once both agents have trained enough to play near-optimally, neither can force a win against a defensively sound opponent, so the majority of games naturally end in a draw rather than a decisive win/loss.

### **Q14. What happens if the number of training episodes is too small?**

With too few training episodes, the agent has not experienced enough game outcomes to form accurate value estimates for most states. It continues to behave close to randomly, frequently misses blocking moves, fails to exploit winning opportunities, and consequently loses more often.

## Screenshots of the RL Tic-Tac-Toe demo:

### Train Agent

Number of Episodes

100

---

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

---

Agent 1 Learning Rate

0.5

---

Agent 2 Explore Probability

0.05

---

Agent 2 Learning Rate

0.5

---

**TRAIN AGENTS**

**SIMULATION**

### Play against agent (Skill level: 100)

You won! Reset game?

|   |   |   |
|---|---|---|
| O | O | X |
|   | X |   |
| X | O |   |

### Agent's Value Function

|       |       |       |
|-------|-------|-------|
| 0     | 50.0% | 50.0% |
| 50.0% | 0     | 50.0% |
| 0     | 0     | 50.0% |

**RESET GAME**

### Train Agent

Number of Episodes

500

---

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

---

Agent 1 Learning Rate

0.5

---

Agent 2 Explore Probability

0.05

---

Agent 2 Learning Rate

0.5

---

**TRAIN AGENTS**

**SIMULATION**

### Play against agent (Skill level: 500)

You won! Reset game?

|   |   |   |
|---|---|---|
| O | X | O |
| O |   | O |
| X | X | X |

### Agent's Value Function

|   |       |       |
|---|-------|-------|
| 0 | 0     | 0     |
| 0 | 25.0% | 50.0% |
| 0 | 50.0% | 0     |

**RESET GAME**

## Train Agent

Number of Episodes

1000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

## Train Agent

Number of Episodes

5000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 1000)

Draw game. Reset game?

X

O

X

X

O

O

O

X

O

Agent's Value Function

0

0

0

0

0

0

0

0

0.0%

RESET GAME

Play against agent (Skill level: 5000)

Draw game. Reset game?

O

X

O

X

O

O

X

O

X

Agent's Value Function

0.0%

0

0

0

0

0

0

0

0

RESET GAME

## Train Agent

Number of Episodes

10000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

## Play against agent (Skill level: 10000)

Draw game. Reset game?



### Agent's Value Function

|   |   |      |
|---|---|------|
| 0 | 0 | 0.0% |
| 0 | 0 | 0    |
| 0 | 0 | 0    |

RESET GAME

## Conclusion

Through repeated self-play, the RL agent gradually transforms from a random, mistake-prone player into a strong, near-optimal one purely by learning from the rewards of winning, losing, and drawing — without ever being told the "correct" move. As training episodes increase, its value function becomes more accurate, so it starts favoring strong opening moves, reliably blocking threats, and eventually setting up its own winning opportunities. Performance improves rapidly in the early stages and then plateaus once learning converges, at which point most games end in a draw because optimal Tic-Tac-Toe play does not allow either side to force a win. This experiment illustrates the core RL idea that intelligent behaviour can emerge purely from trial-and-error interaction with an environment, guided only by a reward signal.